# An Introduction to Reactive Programming and its Semantics

Dr. Ragnar Mogk
Prof. Mira Mezini
Software Technology Group

# Recap

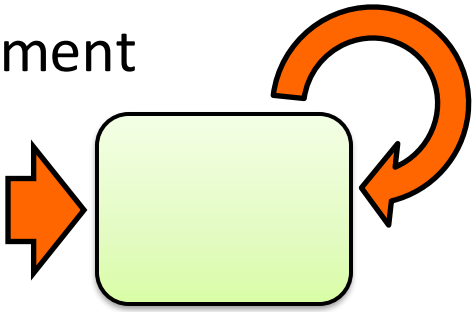
- We discussed the semantics of “obvious” languages
  - Expressions, functions, control, state, objects, …
- The semantics of a program can be very diverse:
  - SQL
  - Prolog
  - HTML
  - Latex
  - Maude
  - …

We see one of such examples that is also having a lot of impact

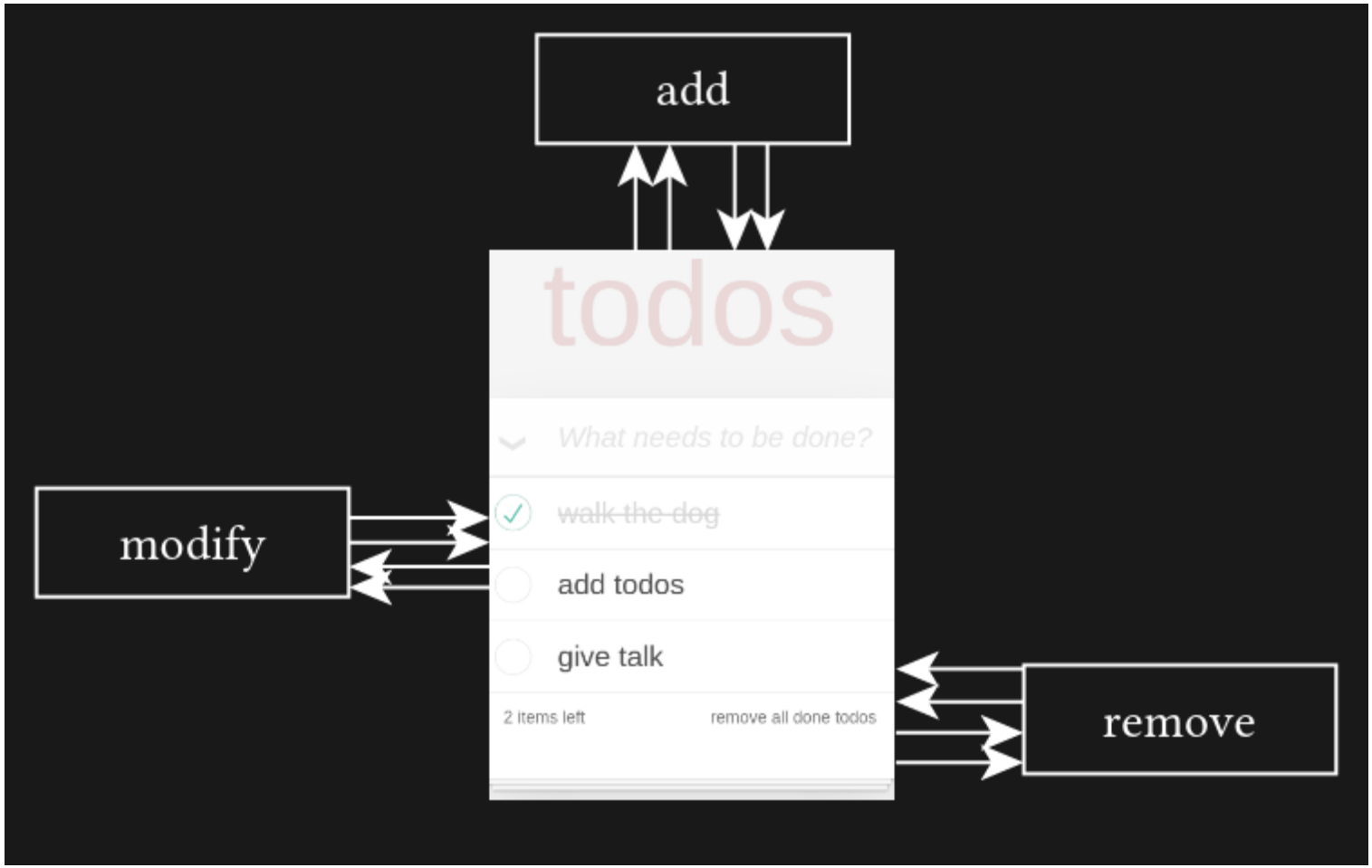
# Software Taxonomy

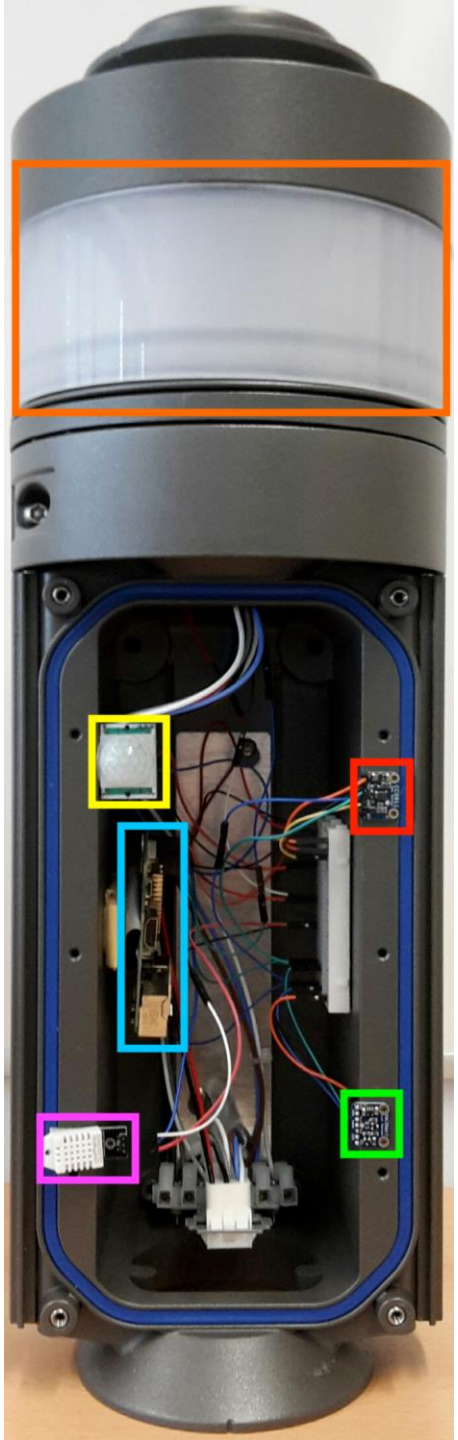
- A **transformational** system
  - Accepts input, performs computation on it, produces output, and terminates
  - Compilers, shell tools, scientific computations

- A **reactive** system
  - Continuously interacts with the environment
  - Updates its state



# Time-changing in reaction to Events





# Reacting to the external (“real”) world

# Reactive Applications

- Examples
  - End user applications
  - (Micro-)Services
  - IoT/Sensors
  - (Stream processing)

- Typical operations
  - Detect events/notifications and react
  - Combine reactions
  - Propagate updates/changes

# Reactive Applications
# Why should we care?

- Event handling:
  - 30% of code in desktop applications
  - 50% of bugs reported during production cycle

# Control Flow

```
var hour : Int = 0
var day : Int = 0
var week : Int = 0
evt tick[Unit] // triggered every hour

evt newDay [Unit] = tick && (() => hour == 0)
evt newWeek [Unit] = tick && (() => day == 0)

tick += () => { hour = (hour + 1) % 24 }
newDay += () => { day = (day + 1) % 7 }
newWeek += () => { week ... }
```

# Reactive Programming

- Reactive applications are extremely common
- Can we design language features
  to specifically address this issue ?

# Reactive Programming

*"Programming language abstractions (techniques and patterns) to develop reactive applications"*

For example, abstractions to:

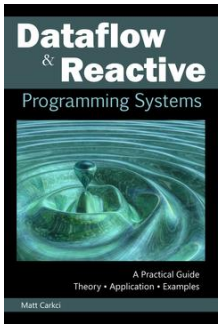
- Represent event streams
- Automatically propagate changes in the state
- Combine events
- …

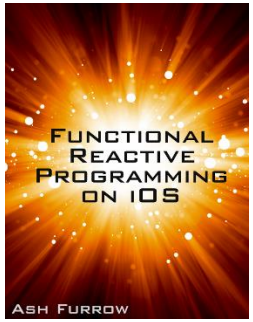
# Reactive Programming (in 2015)

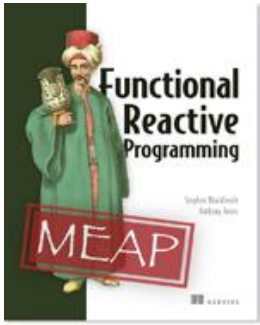
- Haskell: Fran, Yampa
- FrTime, Flapjax, REScala, Scala.react, …

- Angular.js, Bacon.js, Reactive.js, …
- Microsoft Reactive Extensions (Rx)

- Books 2014-15

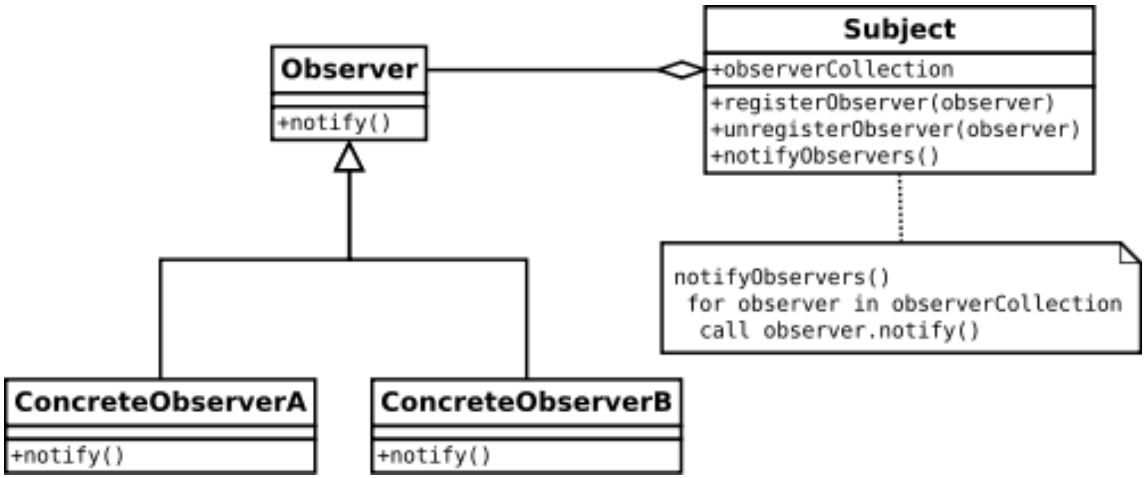


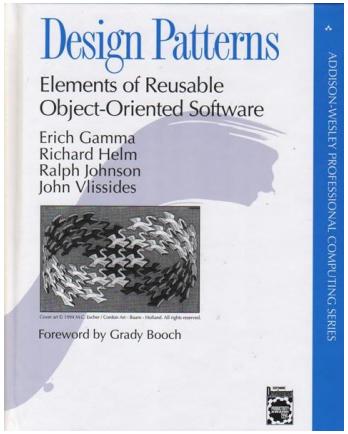


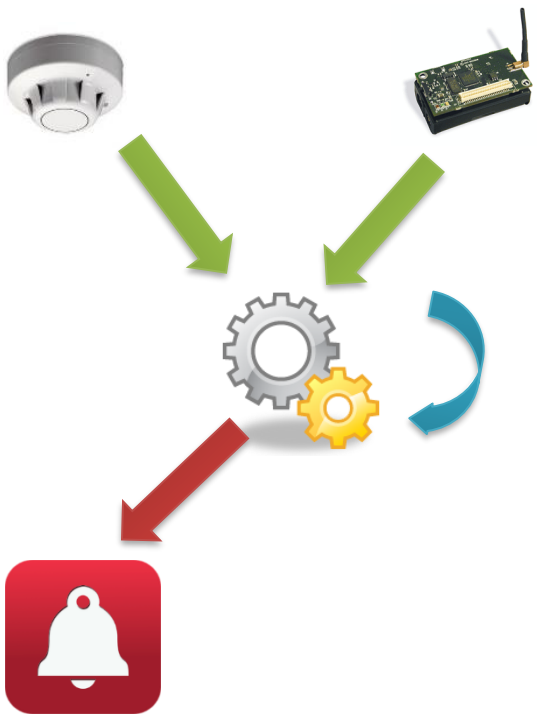
# 2026 Claim:
# Reactive Programming is Mainstream

# The (*good*? old) Observer Pattern



# The (*good*? old) Observer Pattern



```
boolean highTemp;
boolean smoke;

void Init() {
   tempSensor.register(this);
   smokeSensor.register(this);
}

void notifyTempReading(TempEvent e) {
   highTemp = e.getValue() > 45;
   if (highTemp && smoke) {
      alert.start();
   }
}

void notifySmokeReading(SmokeEvent e) {
   smoke = e.getIntensity() > 0.5;
   if (highTemp && smoke) {
      alert.start();
   }
}
```

State

Registration

Callback functions

Control statements

Callback functions

Control statements

# The (*good*? old) Observer Pattern

Events are used to enforce
data dependency **constraints**

highTemp := (temp.value > 45)

# ~~Constraint~~

```
val a: Int
val b: Int
val c = a + b
```

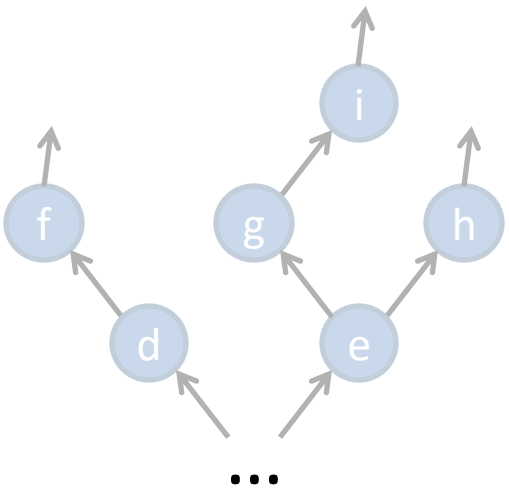
# Reactive Values

```
val a: Var[Int]
val b: Var[Int]
val c = Signal:
    a.value + b.value
```

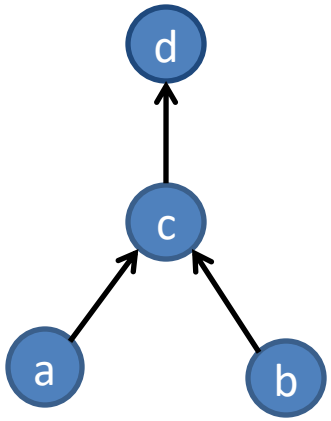
- **Vars**: primitive reactive values
  - Updated by the external World
- **Signals**: reactive expressions
  - The constraints “automatically” enforced

# Reference Model

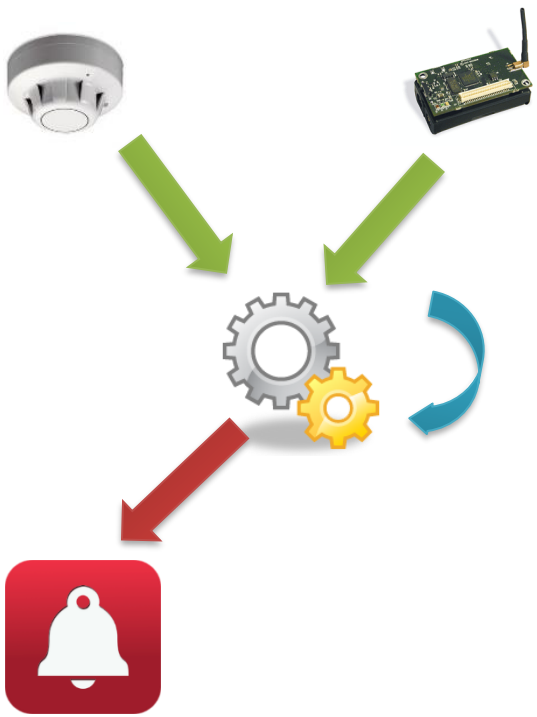
- Change propagation model
- Dependency graph




```
val a = Var(3)
val b = Var(7)
val c = Signal{ a.value + b.value }
val d = Signal{ 2 * c() }
 …
```



# ~~The (*good*? old) Observer Pattern~~



```
val tempSensor: Signal[Int]
val smokeSensor: Signal[Double]

val highTemp = Signal:
   tempSensor.value > 45
val smoke = Signal:
   smokeSensor.value > 0.5

val isAlertState = Signal:
   smoke.value && highTemp.value

isAlertState.observe:
   case true => println(s"FIRE!")
   case false =>
      println(s"fire seems to have stopped ...")
```

# Signals: Examples

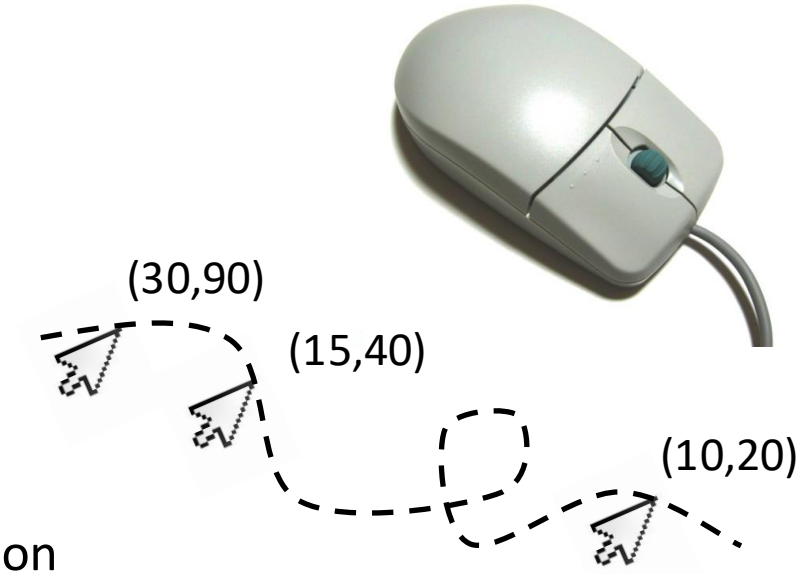
```
val s = Signal { if c.value then a.value else b.value }
```

```
def factorial(n: Int) = ...
val a = Var(0)

val s: Signal[Int] = Signal {   // A signal expression can be any code block
  val tmp = a.value * 2
  val k = factorial(tmp)
  k + 2      // Returns an Int
}
```

# External Events

Mixing signals and events




```
val position: Signal[(Int,Int)] = mouse.position

val shiftedPosition: Signal[(Int,Int)] = Signal{ mouse.position + (10, 10) }

val clicked: Event[Unit] = mouse.clicked

val lastClick: Signal[(Int,Int)] = clicked.snap(position.value)
```

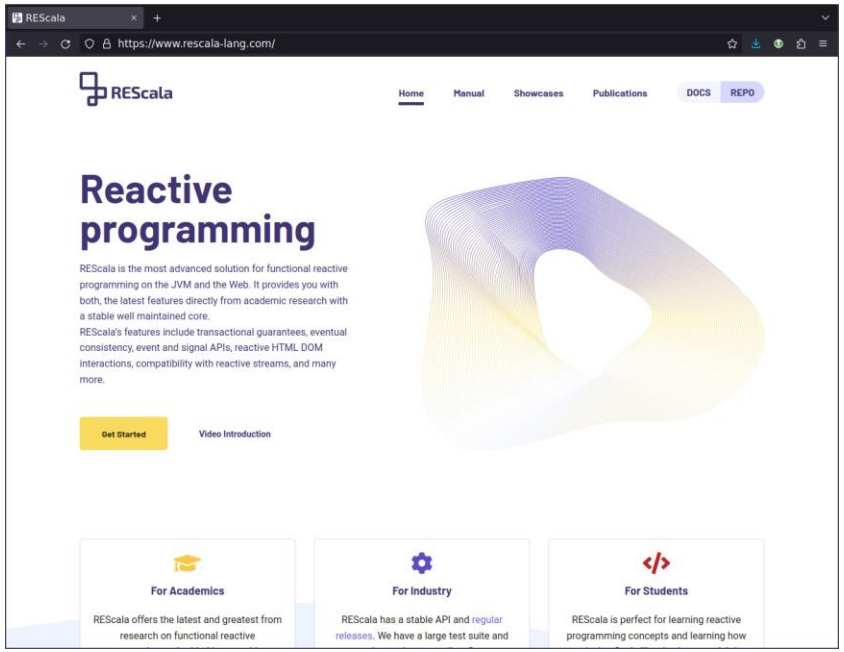
# Reactive Languages

- Functional-reactive programming (FRP) -- Haskell
  - **Time-changing values** as dedicated language abstractions.
    *[Functional reactive animation, Elliott and Hudak. ICFP '97]*

- "More recently":
  - FrTime *[Embedding dynamic dataflow in a call-by-value language, Cooper and Krishnamurthi, ESOP'06]*
  - Flapjax *[Flapjax: a programming language for Ajax applications. Meyerovich et al. OOPSLA'09]*
  - Scala.React *[I.Maier et al, Deprecating the Observer Pattern with Scala.React. Technical report, 2012]*

# *www.rescala-lang.com*

- Documentation
- Quickstart
- Examples / Case studies



# Reactive Languages

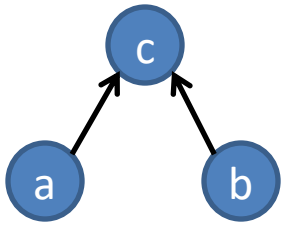
- Easier to understand
  - Declarative style
  - Local reasoning
  - No need to follow the control flow to reverse engineer the constraints
- Dependent values are automatically consistent
  - No boilerplate code
  - No update errors (no updates/update defensively)
  - No scattering and tangling of update code
- Reactive behaviors are composable
  - In contrast to callbacks, which return void

# DETAILS ON THE REACTIVE MODEL

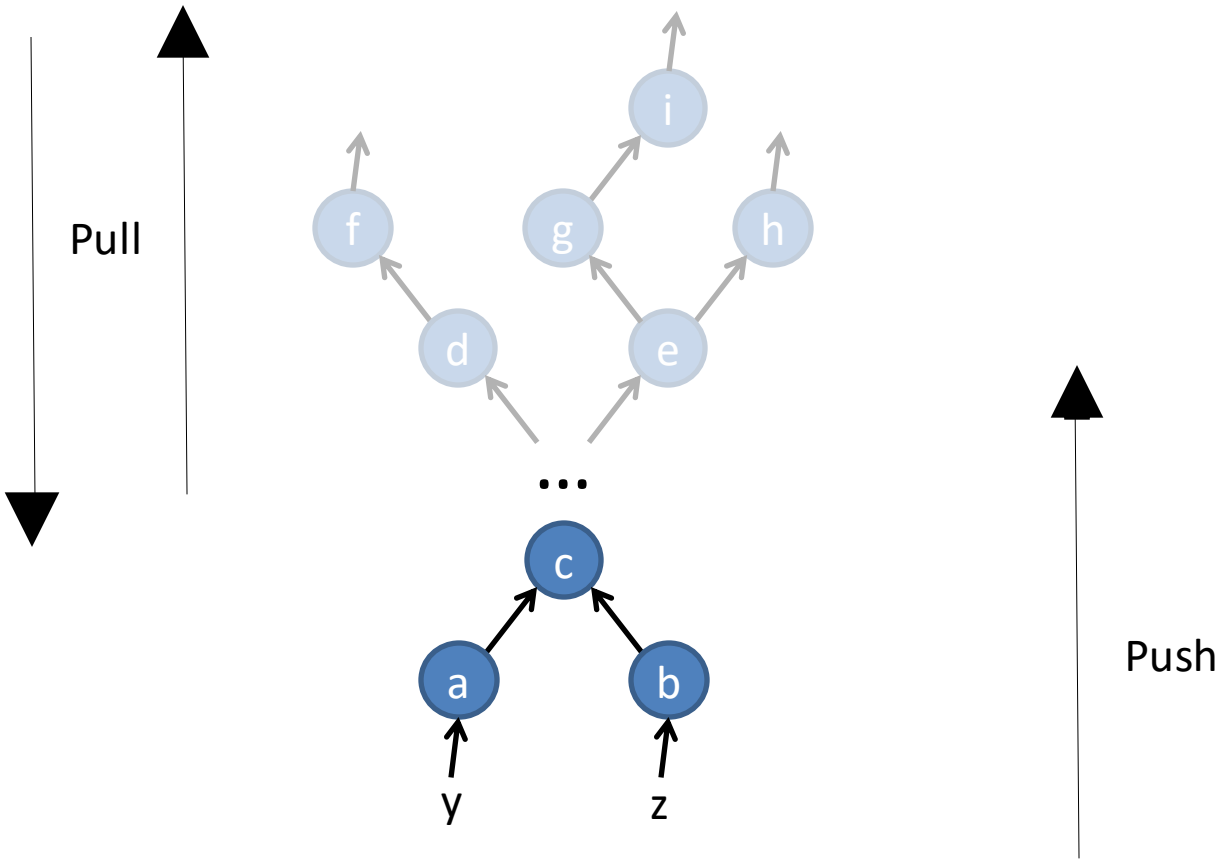
# DSL Implementation

- Building the graph
  - Var(3) -> leaf
  - Var(4) -> leaf
  - “a.value + b.value” saved in a closure
  - Signal{…} -> dependent node

```
val a = Var(3)
val b = Var(4)
val c = Signal {
   a.value + b.value
}
```



# Pull vs. Push Models



# Push vs. Pull

- Push
  - When the external world changes, immediately compute all “derived” changes
  - Great for applications that are mostly idle, and change when something happens
  - Think UIs, sensors, etc
- Pull
  - When you want to “read” a value, pull all changes
  - Great for applications that change “continuously”, think animations or simulations

# Push vs. Pull

- Push
  - Closer to “event based” logic
  - Caching is easy, stop propagation
  - Space for current value, expression closure, dependencies
  - Time to update dependencies if graph is dynamic
- Pull
  - Not clear how to support events in an “easy” way
  - Viewer can decide “sampling rate”
  - Easily move back and forth in time

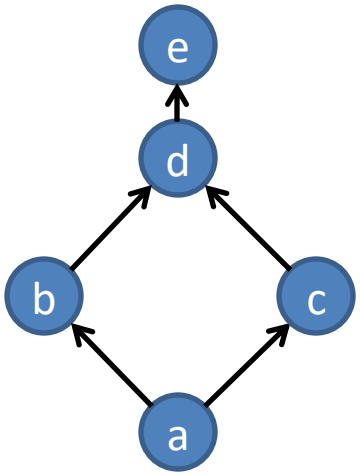
# Glitches

Temporary *spurious* values due to propagation order.

- Update order abdc
- a()=2  b<-4, d<-7, c<-6, d<-10

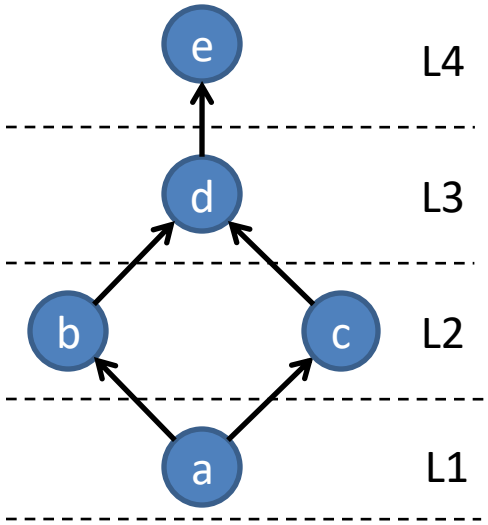
```
val a = Var(1)
val b = Signal{ a.value*2 }
val c = Signal{ a.value*3 }
val d = Signal:
   b.value + c.value
val e = d.changed
```

- Effects:
  - d redundantly evaluated 2 times
  - First value of d has *no meaning*
  - e erroneously fired two times



# Glitch Freedom

- Ensured by updates *in topological order*
  - Nodes are assigned to levels **Ln**
  - Levels are updates in order
  - E.g., "abcde" or "acbde"
- Technical solutions:
  - Priority queue
  - Nodes wait for children

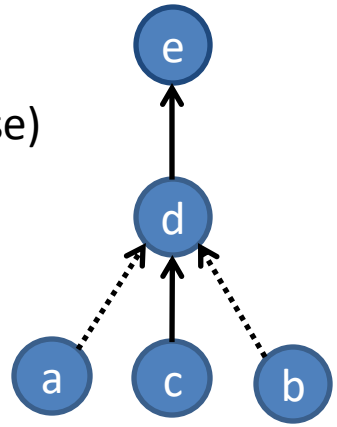


# Dynamic dependencies

- Dependencies based on runtime conditions
  - In case *c==true*, *d* must change:
    - If *a* changes
    - Not if *b* changes
  - *d* depends on *a* or *b* based on current *c*
  - Change dependencies at runtime

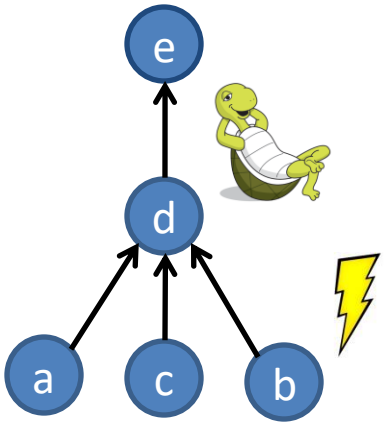
```
val a = ...
val b = ...
val c = Var(false)
val d = Signal:
  if c.value
  then a.value
  else b.value

val e = Signal { 2 * d.value }
```



# (Lack of) Dynamic dependencies

- Easier implementation
- Redundant evaluations
  - d is executed upon b assignments
  - even if the d does not change




```
val a = ...
val b = ...
val c = Var(false)
val d = Signal:
  if c.value
  then a.value
  else b.value

val e = Signal { 2 * d.value }
```

# About Loops

- Reject loops
  - Responsibility to the programmer
  - Loops rejected by the compiler

- Accept loops: which semantics ?
  - Delay to the next propagation round
  - Fix point semantics
    - Time consuming ?
    - Termination ?

```
val x = Signal:
  y.value + 1
val y = Signal:
  x.value + 1
```

# IMPLEMENTING THE INTERPRETER

Thank you !

# QUESTIONS ?

The following Slides were part of the original Lecture in 2015

While they are not presented, they do contain some explanation of potential implementations that may be interesting.

# Syntax: What do we need ?

- Vars
- Signals ?
- Imperative updates to Vars

# Starting point: CFWAE

```
<CFWAE> ::= <num>
          | {+ <CFWAE> <CFWAE>}
          | {- <CFWAE> <CFWAE>}
          | {with {<id> <CFWAE>} <CFWAE>}
          | <id>
          | {fun {<id>} <CFWAE>}
          | {<CFWAE> <CFWAE>}
          | {* <CFWAE> <CFWAE>}
          | {if0 <CFWAE> <CFWAE> <CFWAE>}
```

- What do we need to add?

# Syntax Hint: BCFWAE

```
<BCFWAE> ::= <num>
           | {+ <SCFWAE> <SCFWAE>}
           | {- <SCFWAE> <SCFWAE>}
           | {with {<id> <SCFWAE>} <SCFWAE>}
           | <id>
           | {fun {<id>} <SCFWAE>}
           | {<SCFWAE> <SCFWAE>}
           | {* <SCFWAE> <SCFWAE>}
           | {if0 <SCFWAE> <SCFWAE> <CFWAE>}


           | {newbox <SCFWAE>}
           | {setbox <SCFWAE> <SCFWAE>}
           | {openbox <SCFWAE>}
           | {set <id> <SCFWAE>}
           | {seqn <SCFWAE> <SCFWAE>}
```

# Syntax Hint: RSCFWAE

```
<BCFWAE> ::= <num>
           | {+ <SCFWAE> <SCFWAE>}
           | {- <SCFWAE> <SCFWAE>}
           | {with {<id> <SCFWAE>} <SCFWAE>}
           | <id>
           | {fun {<id>} <SCFWAE>}
           | {<SCFWAE> <SCFWAE>}
           | {* <SCFWAE> <SCFWAE>}
           | {if0 <SCFWAE> <SCFWAE> <CFWAE>}

           | {newvar <SCFWAE>}
           | {setvar <SCFWAE> <SCFWAE>}
           | {curval <SCFWAE>}
           | {set <id> <SCFWAE>}
           | {seqn <SCFWAE> <SCFWAE>}
```

We do not need a syntactic form for signals if we adopt implicit lifting

# Semantics: examples

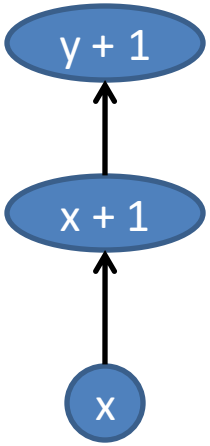
```
Let x = NewVar(1)
  Let y = x + 1
Seqn(
  SetVar(x, 2),
  CurrVal(y)
)
```

```
Let x = NewVar(1)
  Let y = NewVar(2)
    Let z = NewVar(3)
      Let c = If0(x, y, z)
Seqn(Seqn(
   SetVar(x, 0),
   SetVar(y, 4)),
   CurrVal(c)
))))
```

Note lifting of expressions e.g., 1+x

# Semantics: examples

```
Let x = NewVar(1)
  Let y = x + 1
    Let z = y + 1
Seqn(
  SetVar(x, 2),
  CurrVal(z)
)
```



# Time is just a special var

```
Let t0 = CurrVal(Time())
  Let tnow = Time()
 Seqn(
   Wait(100)
   CurrVal(tnow - t0)
 )
```

# Semantics: Summary

- Newvar
  - Create leaf in the tree
  - Nodes in the tree are Vars
- Expression depending on Var
  - Build tree nodes
  - Connect nodes
- Setvar
  - Update leaf value
  - Propagate change in the tree
- Recompute nodes in the tree

This semantics is by far different from what we have seen before (e.g., BCFWAE). In this case, essentially, running a program means building a graph of dependencies and updating it.

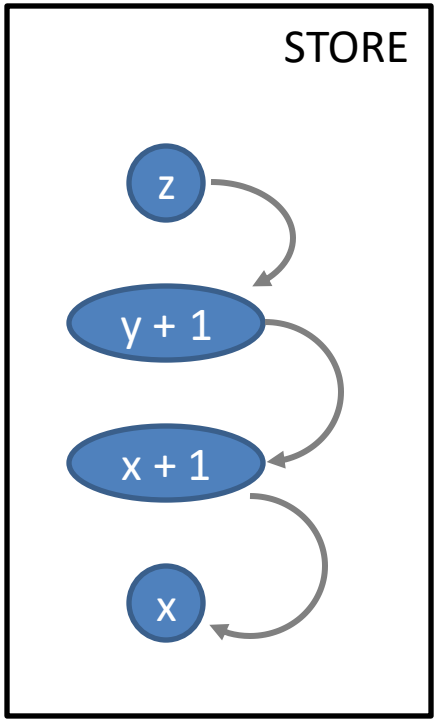
# Another Model: Pull

- Last interpreter was Push
  - Propagate when a change occurs
- What about top-down ?

- Similar to lazy evaluation…
- What happens to the syntax ?

# Pull propagation model

```
Let x = NewVar(1)
  Let y = x + 1
    Let z = y + 1
Seqn(
  SetVar(x, 2),
  CurrVal(z)
)
```




On Demand:

- Store reactive expressions a closures
- “Dependencies” are already available in the store
  - No explicit dependencies
- Setvar just updates the content of x,
  - No propagation
- When CurrVal requests a value, the closure is evaluated

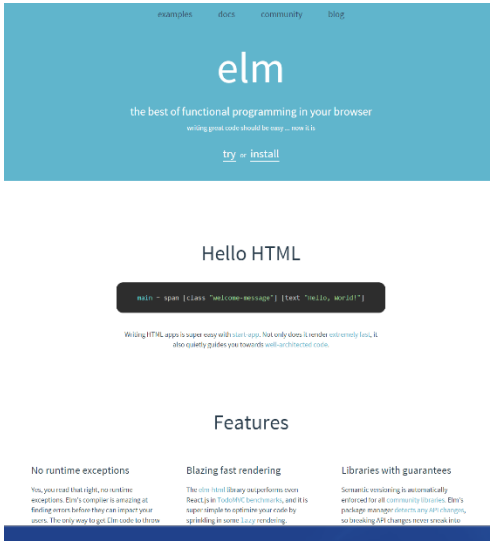
# Push vs. Pull

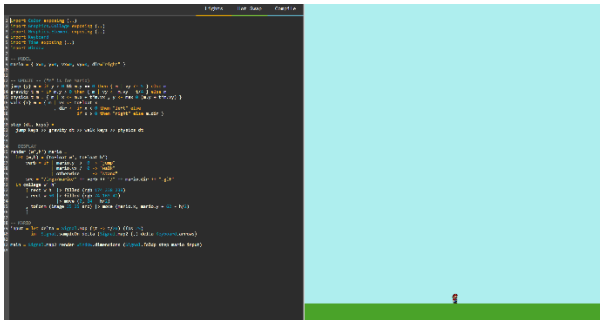
- Any difference in the syntax ?
- Any **observable** difference for the programmer ?

# MORE REACTIVE LANGUAGES

# ELM

- Similar to Haskell
  - Compiles to Javascript
  - Static dependency graph
- Purely functional + static
  - Efficiency
  - Time-travel debugger
- http://elm-lang.org/

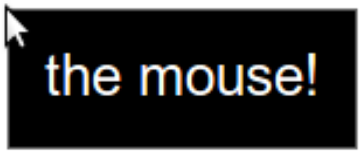


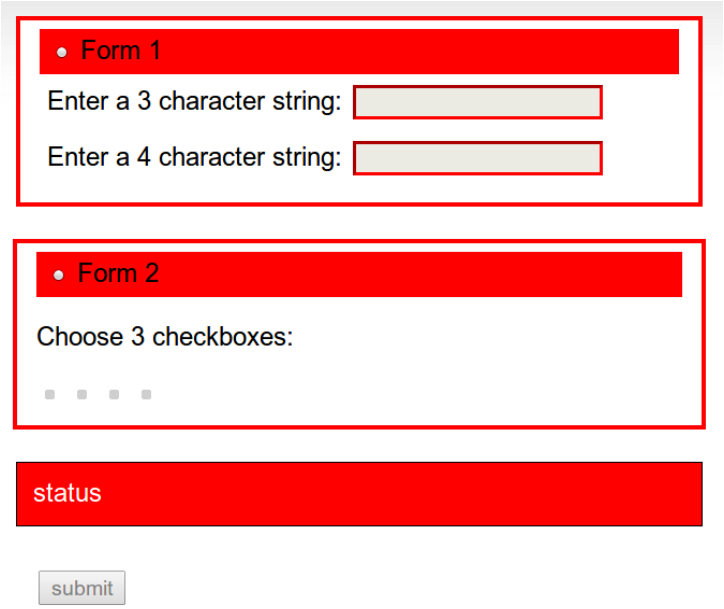
# Flapjax

- Web applications
  - User interaction
  - Events from the server

```
<div id="themouse"
   style={! {color: '#FFFFFF',
            backgroundColor: '#000000',
            position: 'absolute',
             left: mouseLeftB(document),
             top: mouseTopB(document),
             padding: '10px'} !}>
the mouse
</div>
```



```
Enter a 3 character string:
 <input id="textField1"
        style={! {borderColor:
                    (extractValueB('textField1').length === 3 ? 'green' : 'red')}
              !}/>
```



```
<form> <textarea id='theText'> </textarea> <br>
<input type='button' id='saveButton' value='Save Now'></input> </form>
```

```
<script type="text/flapjax">
var textD = $('theText');
var saveD = $('saveButton');
var savedTimeD = $('savedTime');
...
</script>
```

Click 'Save Now' to save the draft. The draft is stored locally, and is not sent to the server. Last Saved:

Save Now

The currently saved draft is:

```
var saveClickE = extractEventE(saveD, 'click');
var autoSaveE = timerE(10000);
var saveE = mergeE(saveClickE, autoSaveE);
var savedTimeE = saveE.mapE(getTime);
insertValueE(savedTimeE, savedTimeD, 'textContent')
```